

CAPACITAÇÃO DOCENTE EM TI

# Frontend Moderno no Mercado

Da arquitetura de monólitos e microserviços aos padrões de renderização, performance no browser, ferramentas de bundling e PWA.

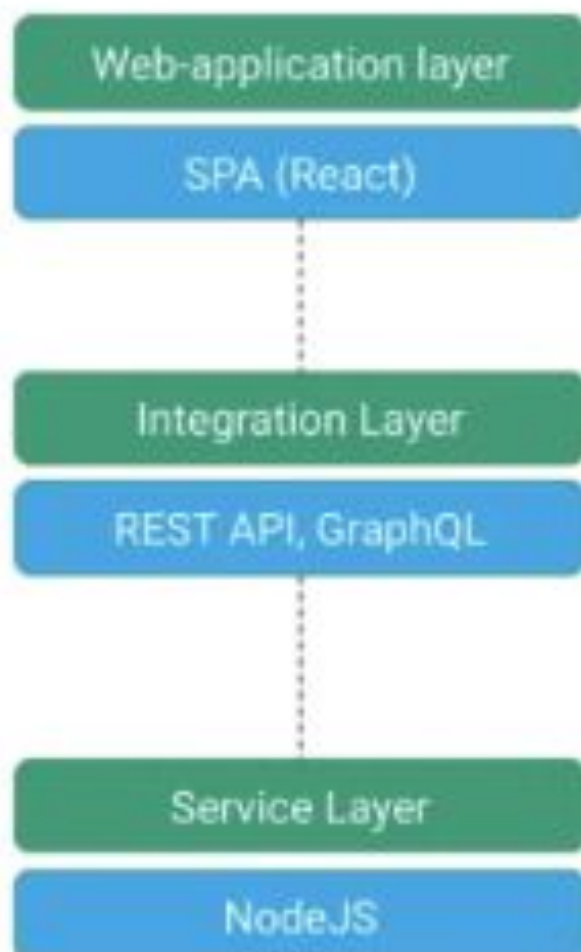
MÓDULO 1

# Arquitetura e Modelos de Render

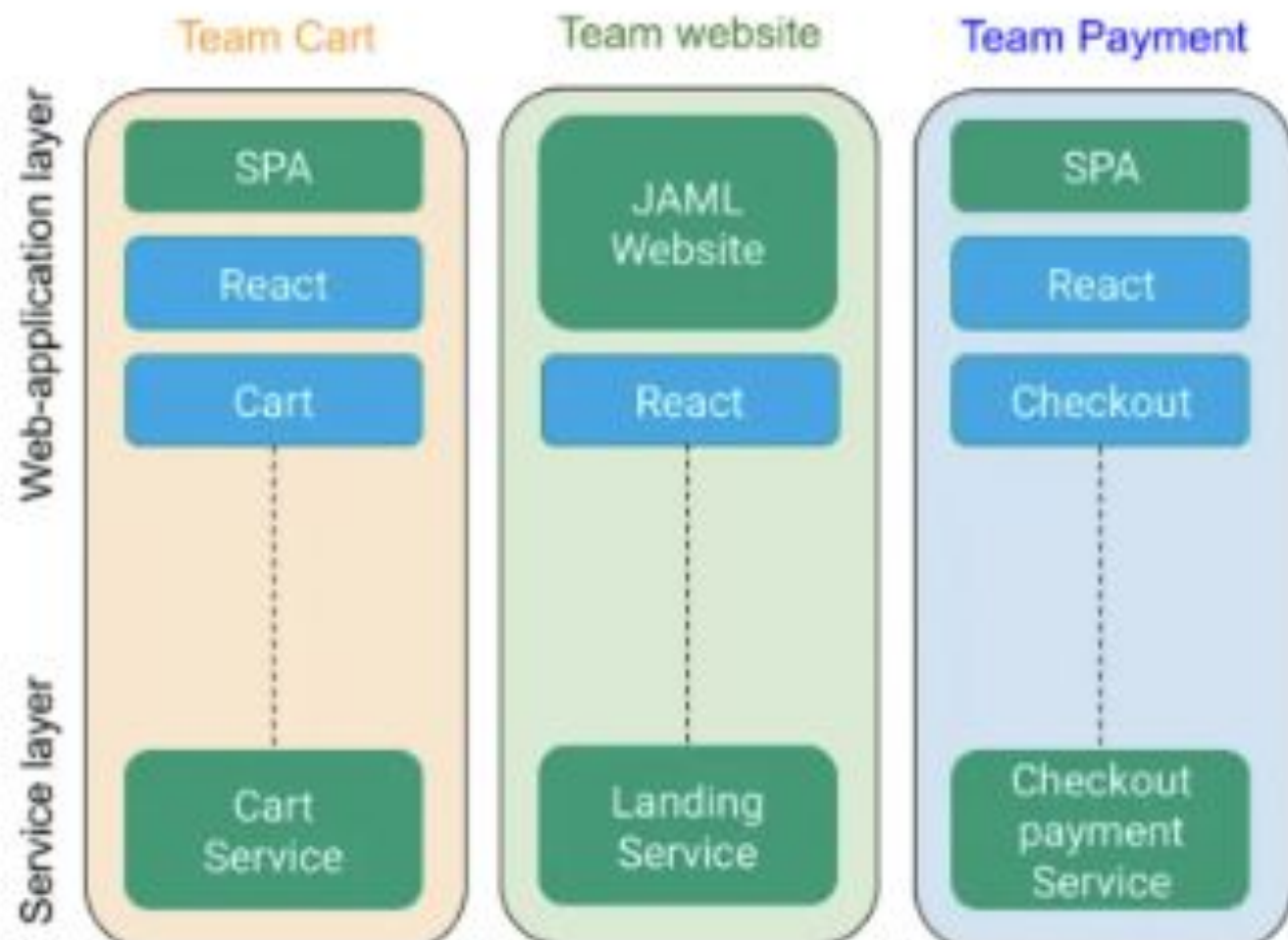
---

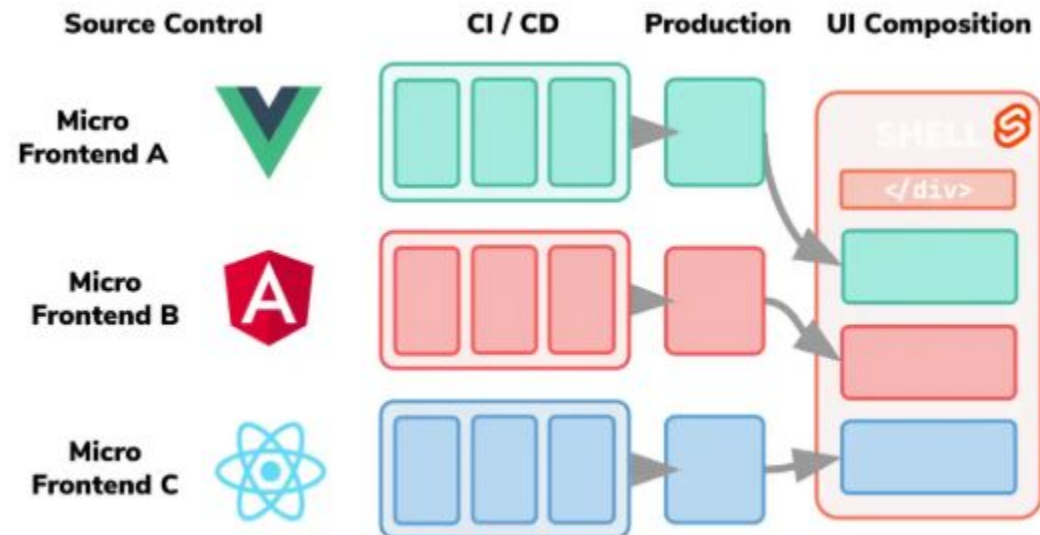
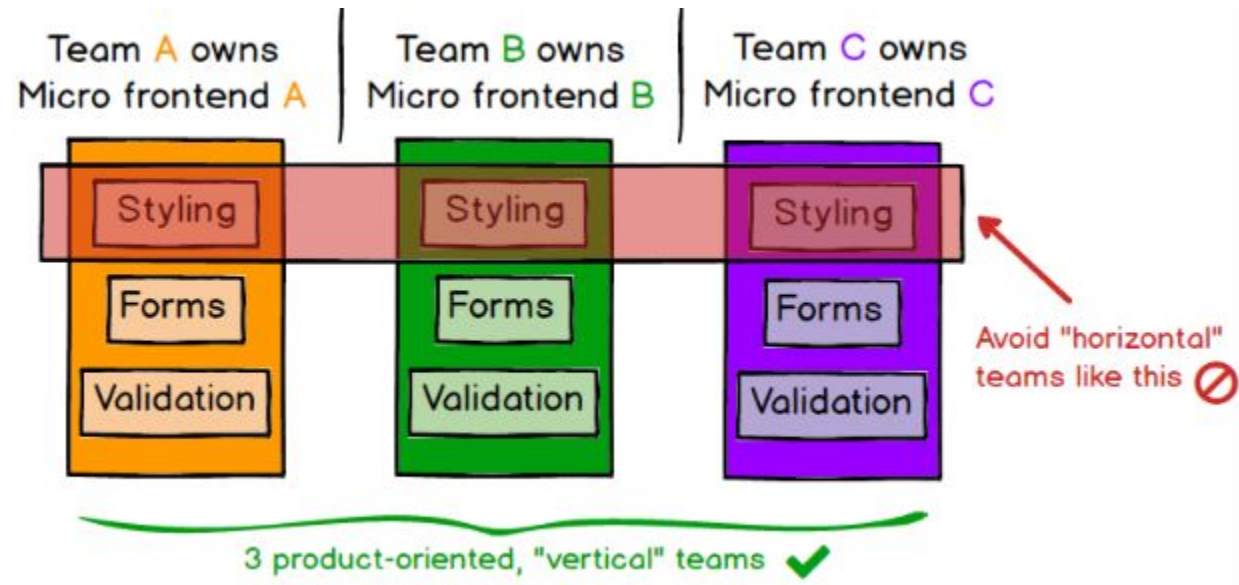
Como alinhar a teoria ensinada na graduação com as decisões arquiteturais reais tomadas pelas grandes empresas de tecnologia.

# Monolithic



# Micro Frontends





# Monólito vs Microfrontends

---

## **Monólito Frontend**

**Vantagens no mercado:** DX simples, facilidade de refatoração, tipagem TypeScript universal e deploy atômico.

**Gargalos de escala:** Em equipes grandes, gera conflito de branches, deploys acoplados e tempo de build excessivo.

**Quando ensinar:** O padrão ideal para startups, projetos de médio porte e MVPs ágeis.

## **Microfrontends (MFE)**

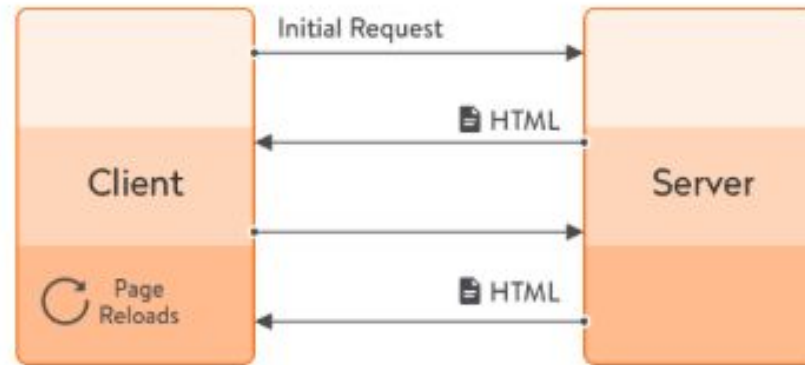
**Vantagens no mercado:** Autonomia total por squads, pipelines de deploy independentes e descentralização de equipes.

**Custos operacionais:** Complexidade em orquestração, duplicação de runtime no cliente e governança de design system.

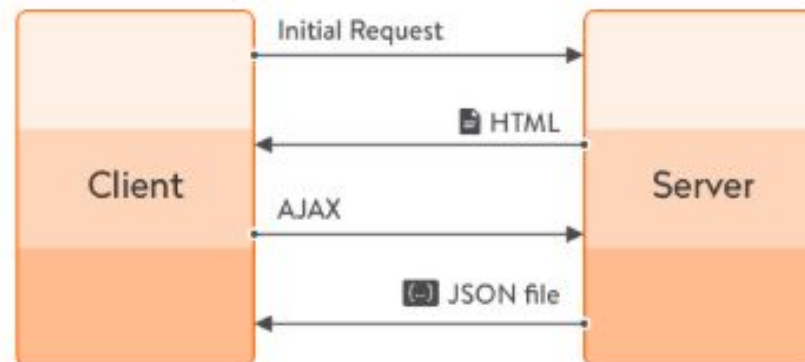
**Tecnologia-chave:** Webpack/Vite Module Federation e isolamento via Web Components.

# SPA vs SSR

## Multi-page app lifecycle



## Single-page app lifecycle



[SPA vs MPA]

# SPA vs SSR e o Impacto em SEO

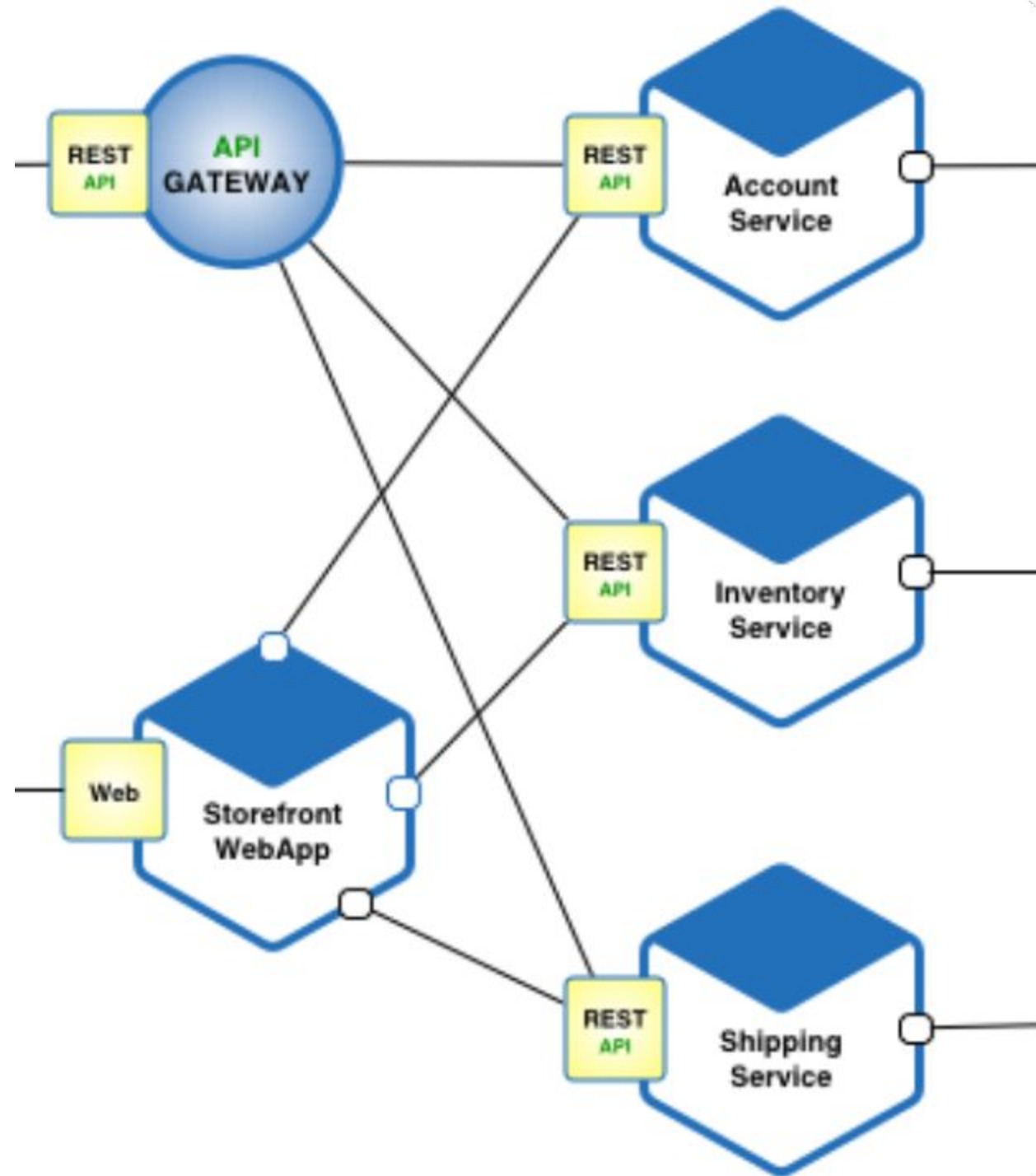
| Critério de Decisão     | SPA (Single Page Application)                     | SSR (Server-Side Rendering)                     |
|-------------------------|---------------------------------------------------|-------------------------------------------------|
| Renderização Primária   | Totalmente no navegador (client-side JS)          | Geração de HTML no servidor por requisição      |
| Indexação de SEO        | Limitada; crawlers podem falhar na execução de JS | Imediata; entrega HTML estruturado completo     |
| Compartilhamento Social | Requer proxies para tags OpenGraph dinâmicas      | Metatags nativas prontas para WhatsApp e redes  |
| Custo de Infraestrutura | Muito baixo (hospedagem em CDN estática)          | Requer runtime de servidor (Node.js / Edge)     |
| Caso de Uso Recomendado | Dashboards internos, portais logados e SaaS       | E-commerce, portais de notícias e landing pages |

# Transição de Paradigma

## O Fim da Era "Apenas Client-Side"

O mercado migrou da discussão simples entre React vs Vue para ecossistemas híbridos. Compreender os limites de renderização define se a empresa escalará sem gargalos de tráfego.

a decisão técnica deve ser guiada por requisitos de negócio: custo de servidor, índice de conversão e velocidade de entrega.





# Ecosystem: Libs vs Meta-Frameworks

---



## React & Vue (Base)

Bibliotecas focadas em interface declarativa. O time precisa escolher manualmente soluções de roteamento, state management e bundler.



## Meta-Frameworks

Next.js, Nuxt e SvelteKit tornaram-se o padrão corporativo por unificar roteamento por pastas, SSR nativo, Server Actions e otimização de imagens.

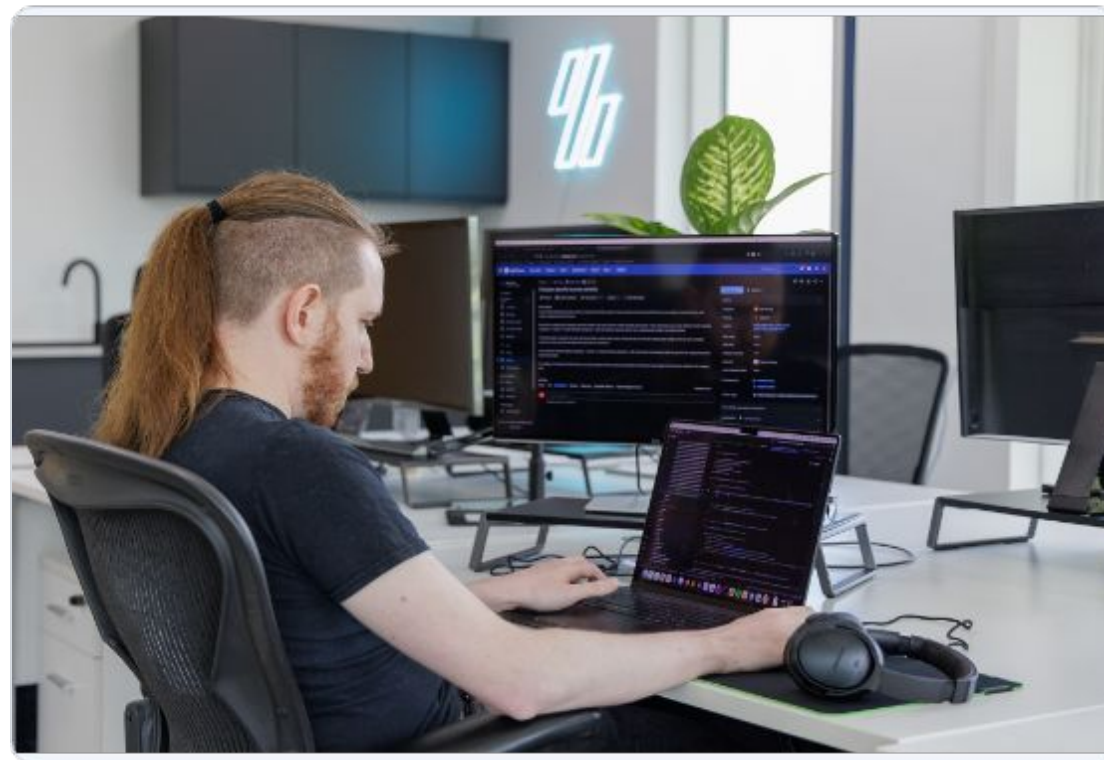


## Gestão de Estado

Transição do Redux clássico para ferramentas leves de estado servidor (TanStack Query/SWR) e estado local atômico (Zustand/Jotai).

# Bundlers Modernos e Otimização

- ⚡ **Evolução das Build Tools:** A substituição gradual de Webpack por Vite (esbuild/Rollup) e Turbopack, trazendo Hot Module Replacement instantâneo.
- ✂️ **Code Splitting por Rota:** Quebra do arquivo de distribuição em chunks sob demanda usando imports dinâmicos (`import()`).
- 🌲 **Tree Shaking Rigoroso:** Eliminação de código morto viabilizada pelo ecossistema ESM (ECMAScript Modules).



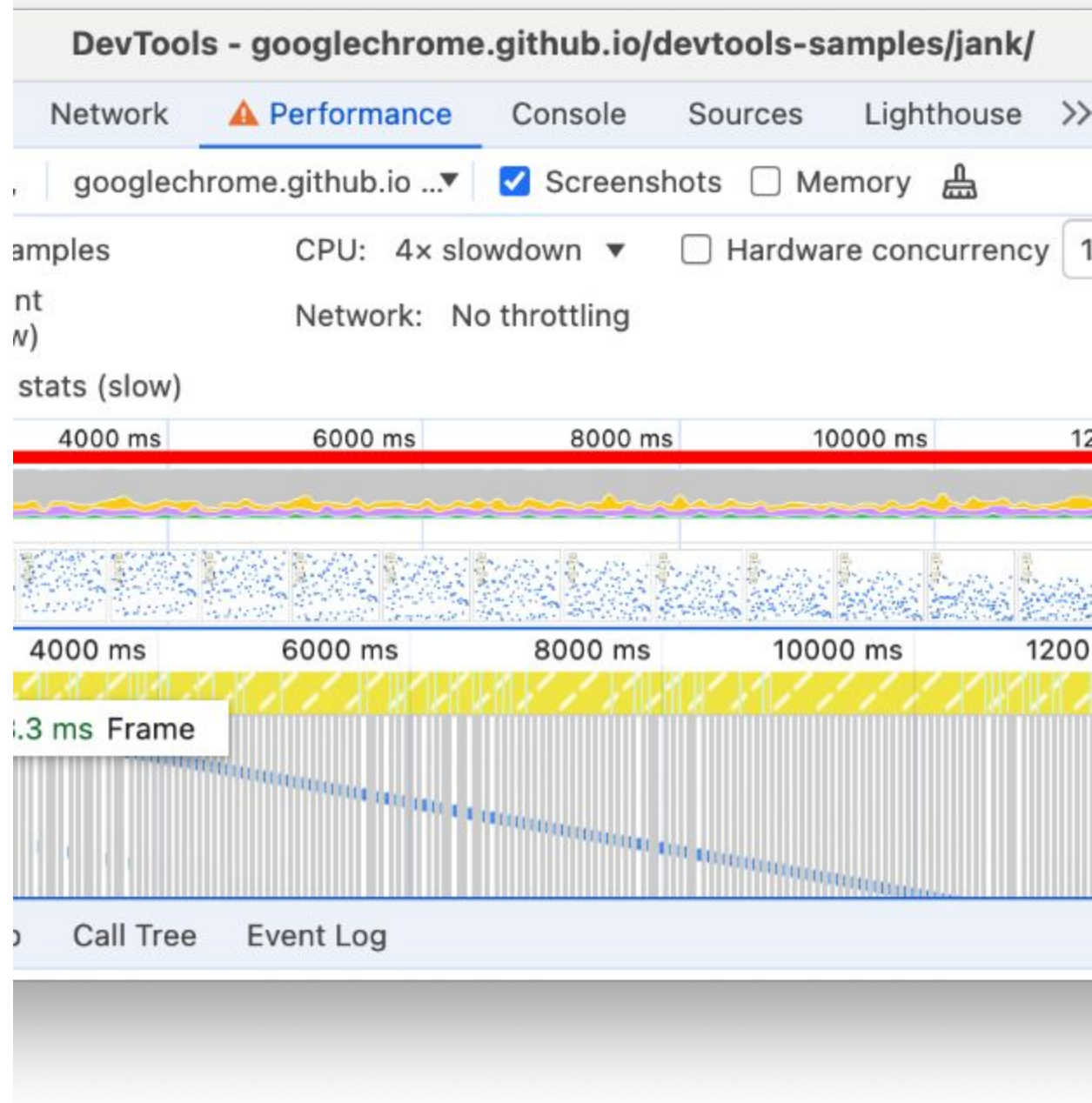
# Diagnóstico com Chrome DevTools

## Profiling Real no Navegador

Auditar performance pelo DevTools é mais eficiente do que suposições teóricas.

A aba **Performance** evidencia Long Tasks (>50ms) que travam a main thread durante execuções pesadas.

A aba **Network** revela waterfalls desnecessários, enquanto a ferramenta **Coverage** identifica scripts e CSS obsoletos carregados na tela inicial.



# Métricas de Performance do Google

---

LCP (Carregamento Visual)

Meta:  $\leq 2.5s$

INP (Interatividade Real)

Meta:  $\leq 200ms$

CLS (Estabilidade Visual)

Meta:  $\leq 0.1$

*A conformidade com as Core Web Vitals afeta diretamente o ranqueamento no Google Search e as taxas de conversão no varejo eletrônico.*

# PWA: Progressive Web Apps

---



**Web App Manifest:** Configura metadados para instalação na tela inicial, ícones e modo de exibição standalone.



**Service Workers:** Scripts que rodam em background, interceptando requisições e operando de forma offline.



**Estratégias de Cache:** Implementação de padrões Stale-While-Revalidate, Cache-First e Network-First.



**Realidade de Mercado:** Excelente tração em Android e ecossistemas B2B; restrições históricas no iOS/WebKit.



**Interface responsiva de Progressive Web App em desktop e celular**

# Perguntas & Dinâmica

✓ Lab Chrome DevTools

✓ Análise de Bundle

✓ Decisão de MFE vs Monólito

✓ Auditoria de SEO Técnico